

Guía 3- Ing. del software II

Ejercicio 2. (En referencia al problema de los filósofos [Cap. 6, Concurrency, Magee & Kramer 1999].) Una solución al problema de los filósofos permite que solo cuatro filósofos se sienten a la mesa al mismo tiempo. Especifique el proceso BUTLER que, al componerlo con el modelo de la sección 6.2 permite que un máximo de cuatro filósofos ejecuten el evento `sitdown` sin que antes ocurra un evento `arise`. Muestre que este sistema es libre de deadlock.

Ejercicio de LST.

Ejercicio 3. Demuestre que la intersección de propiedades de safety es una propiedad de safety.

Sea Σ un lenguaje y P_1, P_2 propiedades de safety de Σ .

Recordemos que

$$\forall \sigma \notin P_1 \quad \exists i \forall \beta \in [\dots i] \beta \notin P_1$$

$$\forall \sigma \notin P_2 \quad \exists i \forall \beta \in [\dots i] \beta \notin P_2$$

si $\sigma \notin P_1$ o $\sigma \notin P_2$ claramente $\sigma \notin P_1 \cap P_2$

$$\text{como } \sigma \notin P_1, P_2 \quad \exists i \forall \beta \in [\dots i] \beta \notin P_1$$

$$\text{y } \exists i \forall \beta \in [\dots i] \beta \notin P_2$$

$$\text{Es decir } \exists i \forall \beta \in [\dots i] \beta \notin P_1 \cap P_2$$

ya que si $\sigma \in [\dots i] \beta \notin P_2$ entonces $\sigma \in [\dots i] \beta \notin P_1 \cap P_2$

y si $\sigma \in [\dots i] \beta \notin P_1$ entonces $\sigma \in [\dots i] \beta \notin P_1 \cap P_2$

Ejercicio 4. ¿Es cierto que la intersección también preserva las propiedades liveness?

No, de hecho sea $\Sigma = \{a, b\}$

$$P_1 = (a+b)^* a^w$$

$$P_2 = (a+b)^* b^w$$

ambas son propiedades de liveness pero $P_1 \cap P_2 = (a+b)^*$ y no es una propiedad.

Ejercicio 5. Encuentre alguna propiedad que sea simultáneamente de safety y de liveness.

Sea $\Sigma = \{a, b\}$

$$P = (a+b)^w$$

P propiedad de safety y liveness

$$\forall \alpha \in \Sigma^* \exists \beta \in \Sigma^w \text{ t.q. } \alpha\beta \in P$$

y como $\forall \alpha \in \Sigma^w \alpha \in P$ tenemos que $\forall \alpha \notin P \exists \beta \in \Sigma^w \text{ t.q. } \alpha\beta \in P$
↳ rango vacío.

Ejercicio 6. Con referencia al ejercicio anterior, demuestre que esa propiedad que encontró es única.

Es única con respecto a las tazas que puede producir, por ejemplo
 $P_1 = (arb)^* (arb)^{\omega}$ $P_2 = (arb)^{\omega}$

Es claro que ambas propiedades son las mismas.

Para demostrar esto primero definamos los conjuntos pref y closure .

Sea Σ un lenguaje y $\sigma \in \Sigma^{\omega}$, definimos

$$\text{pref}(\sigma) = \{ \sigma' \in \Sigma^* \mid \sigma' \text{ es un prefijo finito de } \sigma \}$$

Aí si $\sigma = \alpha_1 \alpha_2 \alpha_3 \dots$ $\text{pref}(\sigma) = \{ \epsilon, \alpha_1, \alpha_1 \alpha_2, \dots \}$.

Sea P una propiedad

$$\text{pref}(P) = \bigcup_{\sigma \in P} \text{pref}(\sigma)$$

Definimos closure como

$$\text{closure}(P) = \{ \sigma \in \Sigma^{\omega} : \text{pref}(\sigma) \subseteq \text{pref}(P) \}.$$

P es de safety si $\text{closure}(P) = P$. Píbebo. Claramente $P \subseteq \text{closure}(P)$.

Supongamos que $\text{closure}(P) \neq P$. Entonces existe $\sigma \in \text{closure}(P) \setminus P$ como P es de safety entonces existe un $\hat{\sigma} = \sigma[..*i*]$ (izo (prefijo malo) tal que $\forall \beta \in \Sigma^* \hat{\sigma}\beta \notin P$. Como $\sigma \in \text{closure}(P)$ entonces por def. $\hat{\sigma} \subseteq \text{pref}(P)$ entonces existe una palabra $\sigma' \in P$ tal que $\hat{\sigma}v \in P$ con $v \in \Sigma^*$, pero esto contradice la hipótesis de que P es de safety.

Por otro lado si P es de liveness si toda palabra finita puede ser extendida a una palabra infinita que satisfaga P , es decir que $\text{pref}(P) = \Sigma^*$.

Ahora supongamos que P es una propiedad de safety y liveness, como vimos antes por ser de liveness $\text{pref}(P) = \Sigma^*$, por ende se sigue que $\text{closure}(P) = \Sigma^{\omega}$. Por otro lado al ser de safety $\text{closure}(P) = P$ por ende igualando las dos últimas ecuaciones tenemos que $P = \Sigma^{\omega}$.

Ejercicio 7. Demuestre o refute lo siguiente:

"El complemento de una propiedad de safety es necesariamente una propiedad de liveness".

Falso. Contraejemplo:

Sea $\Sigma = \{a, b\}$.

$$P = b(atb)^w$$

P es una propiedad de safety y es satisfecha sólo por aquellas palabras que comienzan con b .

$$P^c = a(atb)^w$$

P^c es el complemento de P , acepta sólo las palabras que comienzan con a . Claramente P^c no es una propiedad de liveness ya que $b \in \Sigma^*$ y no existe $\beta \in \Sigma^w$ tal que $b\beta \in P^c$.

Ejercicio 8. Considere el alfabeto $\Sigma = \{a, b\}$. Determine si las siguientes propiedades son de *safety*, *liveness*, ambas, o ninguna. Justifique la respuesta:

- | | | |
|-----------------------|------------------------------------|----------------------|
| i. a^*b^ω | iii. $a^*b^+a^\omega$ | v. $(ab)^\omega$ |
| ii. $(b+a)^+b^\omega$ | iv. $(a+b)^*(a^\omega + b^\omega)$ | vi. $(ab)^*a^\omega$ |

i. No es de *safety* ya que:

$a^\omega \notin P$ sin embargo no hay prefijo malo (son todas a 's finitas, las cuales se pueden extender con b 's infinitas y así satisfacen P)

No es de *liveness* ya que:

$ba \in \Sigma^*$ y no existe $\beta \in \Sigma^\omega$ tal que $ba\beta \in P$.

ii. No es de *safety* ya que:

$a^\omega \notin P$ sin embargo no hay prefijo malo (son todas a 's finitas, las cuales se pueden extender con b 's infinitas y así satisfacen P)

Es de *liveness* $\text{pref}(P) = \Sigma^+$, se pueden armar todas las palabras de Σ^+ con el prefijo, a cualquier palabra de Σ^* le puedo agregar infinitas b 's y va a pertenecer a P .

iii. No es de *safety* ya que:

$a^\omega \notin P$ sin embargo no hay prefijo malo (son todas a 's finitas, las cuales se pueden extender con una o más b 's y a 's infinitas y así satisfacen P)

No es de *liveness* ya que:

$bab \in \Sigma^*$ y no existe $\beta \in \Sigma^\omega$ tal que $bab\beta \in P$.

iv. No es de *safety*:

$ababab\dots \notin P$ sin embargo no hay prefijo malo (en cualquier punto se puede extender con a 's o b 's infinitas y así satisfacen P)

Es de *liveness*, con el prefijo $(a+b)^*$ se pueden generar todas las palabras de Σ^* , luego con $\beta = a^\omega$ o $\beta = b^\omega$ se extiende la palabra

y satisface P .

v. Es de safety, cualquier palabra que no este en P va a tener dos a's o dos b's consecutivas (o empieza por b), si empieza por b ese es el prefijo malo, caso contrario va a ser el i donde aparezca la segunda a o b consecutiva.

No es de liveness, $b \in \Sigma^*$ y $\nexists \beta \in \Sigma^w : b\beta \in P$.

vi. No es de safety, $ab^w \notin P$ pero no hay prefijo malo (se extiende con a^w y satisface P)

No es de liveness, $ba \in \Sigma^*$ y $\nexists \beta \in \Sigma^w : ba\beta \in P$.

Ejercicio 9. Para cada propiedad P del Ejercicio 8 que no sea de safety extiéndala agregando el menor conjunto de trazas de manera que lo sea. Use expresiones ω -regulares. Justifique su respuesta.

i. $a^+ b^\omega$

El único problema era que $a^\omega \notin P$, entonces:

$$P = a^*(a^\omega + b^\omega)$$

ii. $(b+a)^+ b^\omega$

El problema es cualquier palabra que pueda ser generada con $(b+a)^+$ ya que se pueden repetir infinitamente (sin incluir a b^ω)

$$P = (b+a)((b+a)^* b)^\omega$$

iii. $a^+ b^+ a^\omega$

El problema son las siguientes palabras: $a^\omega, ab^\omega, b^\omega$.

$$P = a^+(b^+ a^\omega + b^\omega + a^\omega)$$

iv. $(a+b)^*(a^\omega + b^\omega)$

El prefijo $(a+b)^*$ permite generar cualquier palabra, así que P debe aceptar todas las palabras, luego

$$P = (a+b)^\omega$$

v. $(ab)^+ a^\omega$

El único problema es ab^ω

$$(ab)^+(a^\omega + (ab)^\omega)$$

No estoy muy segura de estos

Ejercicio 10. Para cada propiedad P del Ejercicio 8 que no sea ni de safety ni de liveness, dé una propiedad de safety S y una de liveness L tal que $S \cap L = P$. Justifique su respuesta.

$$\begin{aligned} \text{i. } a^* b^w & \quad L = a^* b^w \cup (a+b)^w \setminus a^*(a^w + b^w) \\ S = a^*(a^w + b^w) & \quad L = a^* b^w \cup (ba+b)(a+b)^w \\ & \quad L = (ba+b+a^*b)(a+b)^w \end{aligned}$$

$$\begin{aligned} S \cap L & \\ = a^*(a^w + b^w) \cap (ba+b+a^*b)(a+b)^w & \\ = a^* b^w & \\ = P & \end{aligned}$$

$$\begin{aligned} \text{iii. } a^* b^+ a^w & \quad L = a^* b^+ a^w \cup (a+b)^w \setminus a^*(b^+ a^w + b^w + a^w) \\ S = a^*(b^+ a^w + b^w + a^w) & \quad L = a^* b^+ a^w \cup bab(a+b)^w \end{aligned}$$

$$\begin{aligned} S \cap L & \\ = a^* b^+ a^w & \end{aligned}$$

$$\begin{aligned} \text{v. } (ab)^* a^w & \quad L = (ab)^* a^w \cup (a+b)^w / (ab)^*(a^w + (ab)^w) \\ S = (ab)^*(a^w + (ab)^w) & \quad L = (ab)^* a^w \cup (a+b)^*(ba+b)^w \\ & \quad L = (a+b+ab)^*(ba+ab+a)^w \end{aligned}$$

$$\begin{aligned} S \cap L & \\ = (ab)^*(a^w + (ab)^w) \cap (a+b+ab)^*(ba+ab+a)^w & \\ = (ab)^* a^w & \end{aligned}$$

Ejercicio 11. Dé ejemplos de trazas que violan la siguiente propiedad:

property PS = $(a \rightarrow (b \rightarrow PS \mid a \rightarrow PS) \mid b \rightarrow a \rightarrow PS)$.

$a, b, b, b, b, \dots$

$b, b, b, b, \dots$

$a, a, b, b, \dots$

a, b, b, a, a, b, b, b

Ejercicio 13. Un ascensor tiene una capacidad máxima de 10 personas. En el modelo del sistema de control del ascensor, los pasajeros que ingresan al ascensor se señalan con un evento `enter` y los que salen se señalan con un evento `exit`. Especifique una propiedad de safety en FSP que cuando sea compuesta con el ascensor verifique que el sistema nunca permita que el ascensor controlado por este exceda los diez ocupantes.

Ejercicio 14. (Ej. 7.6 del *Concurrency, State Models and Java Programs, 2nd ed.*, Magee & Kramer 2006.) Dos vecinos conflictivos tiene separado sus terrenos por un campito con moras. Ellos acordaron que se iban a permitir entrar al campito para recoger moras, pero bajo la condición de que a lo sumo uno de ellos está a la vez en el campito. Luego de varias negociaciones llegaron al acuerdo del siguiente protocolo.

Cada vez que uno de los vecinos desea entrar al campito, iza su propia bandera. Si este vecino ve que la bandera del otro ya estaba izada, arría la bandera propia e intenta nuevamente. Si, en cambio, la bandera del otro vecino no está izada, ingresa al campito y recoge las moras que desea. Una vez que haya culminado sale del campito y arría su bandera.

Modele este algoritmo para dos vecinos, `n1` y `n2`. Especifique la propiedad de *seguridad* requerida para el campito y verifique que asegure efectivamente el acceso en exclusión mutua de los vecinos. Especifique las propiedades de *progreso* requeridas para que ambos vecinos puedan recoger moras provisto que se encuentran bajo una *estrategia de scheduling equitativa*. ¿Existe alguna circunstancia adversa en la cual los vecinos no puedan progresar? ¿Qué ocurre si alguno de los vecinos es egoísta? (En la página 158, el libro da una ayuda para modelar las banderas.)